

Steam Lite Simulator

CPIT 305 Advanced Programming - Final Project Report

King Abdulaziz University | Faculty of Computing and Information Technology

Team Information

#	Student Name	Student ID
1	Name: Abdullah Salah	ID:2336019
2	Name: Abdulrahman Abualenain	ID: 2339030
3	Name: Mohamad Alghamdi	ID: 2338595

Project Overview

Steam Lite is a multi-user game library platform inspired by digital distribution systems such as Steam. The application allows users to register, log in, browse a game catalog, view game details, check wallet balance, purchase games, and download game files from a central server.

Problem Statement

The project solves the problem of managing a shared digital game library where multiple users can access catalog information and download game assets at the same time. It demonstrates how a Java application can combine networking, concurrency, file processing, and persistent database storage in one complete system.

Main Features

- User registration and login with SHA-256 password hashing.
- Central game catalog loaded from resource metadata files.
- TCP client-server communication using serialized Message objects.
- Concurrent client handling through a fixed thread pool.
- Game purchasing, wallet balance deduction, and download tracking.
- Chunked binary file transfer from the server to the client.
- SQLite database persistence for accounts, games, and download history.
- Transaction and event logging to steamlite.log.

Technical Architecture

Layer	Class	Purpose
Client	GameClient	Command-line interface that connects to the server and sends user commands.
Protocol	Message	Serializable object used to exchange request, response, and binary chunk data.
Server	GameServer	Listens on TCP port 9090 and dispatches each client session to the thread pool.

Session Handler	ClientHandler	Processes login, catalog, balance, and download requests for one client.
Database Layer	DatabaseManager	Creates schema and performs synchronized JDBC operations.
File Loader	GameLoader	Reads game metadata files and prepares binary asset files.

Course Requirements Mapping

Requirement	Status	Evidence
IO Streams	Met	Reads text metadata, creates binary assets, streams files in chunks, writes logs.
Multithreading	Met	Uses ExecutorService fixed thread pool and synchronized database methods.
Networking	Met	Uses ServerSocket, Socket, ObjectInputStream, and ObjectOutputStream.
Database Integration	Met	Uses SQLite JDBC, schema creation, CRUD operations, transactions, and foreign keys.
Exception Handling	met	Uses try-with-resources, catch blocks, logging, and graceful shutdown handling.
Testing	met	Custom unit tests exist, but test isolation should be fixed before final submission.

Database Design

The system uses SQLite as the database engine through JDBC. The schema follows the suggested Steam Lite project structure and keeps persistent records for games, accounts, and downloads.

Table	Important Fields	Purpose
Games	id, title, developer, genre, price, description, file_path	Stores game catalog metadata.
Accounts	id, username, password_hash, date_of_birth, balance	Stores registered users and wallet balance.
Downloads	account_id, game_id, download_count, last_downloaded	Tracks ownership and repeated downloads.

Implementation Details

- The server starts by connecting to the database, creating the schema if needed, and seeding the game catalog.
- Each client connection is handled by a ClientHandler task submitted to a fixed-size thread pool.
- Database methods are synchronized to protect shared access from multiple client threads.
- Game downloads are sent as repeated DOWNLOAD_CHUNK messages until DOWNLOAD_COMPLETE is received.
- The client writes downloaded game bytes into the local downloads directory.

Testing and Reliability

- Password hashing tests verify hash length, repeatability, and hex formatting.
- Database tests cover game insertion, duplicate prevention, account registration, login, balance deduction, and download counts.
- Game retrieval tests verify listing and lookup behavior.
- Recommended improvement: make DatabaseManager read the test database property so tests run against an isolated in-memory database.

How to Run the Project

1. Open a terminal in the SteamLite project folder.
2. Start the server using: `java -cp "target\classes;lib\*" steamlite.server.GameServer`
3. Open a second terminal in the same folder.
4. Start the client using: `java -cp "target\classes;lib\*" steamlite.client.GameClient`
5. Use commands such as register, login, list, detail, download, balance, help, and quit.

Demo Plan

6. Show the project structure and explain the main packages: client, server, db, model, util, and tests.
7. Open the SQLite database and show the Games, Accounts, and Downloads tables.
8. Run the server and connect one client.
9. Register or log in, list games, view details, and download a game.
10. Explain how the server handles multiple clients using the thread pool.

Known Limitations and Improvements

- The user interface is command-line based; a graphical interface could improve usability.
- Tests should use a separate in-memory database instead of the real steamlite.db file.
- The packaged JARs should be rebuilt with the correct Windows SQLite native library included.
- Git repository history should be prepared before final submission to satisfy version control requirements.

Conclusion

Steam Lite satisfies the main CPIT 305 final project requirements by integrating IO Streams, multithreading, networking, and database interaction into one working Java application. The project is a strong match for the suggested Multi-User Game Library idea and is suitable for demonstration after the remaining packaging, testing, and version control improvements are addressed.